AuraSoft API Test Tool with an AI layer

System overview: architecture, what is built, how it works, how good the AI is, and what is next

Status	**All planned phases complete (0–6)** + eval sets · Claude verified live
Date	25 October 2026
Repository	github.com/levan-petrosiani/Aurasoft-Monitoring · branch `main`
Stack	Python 3.11+ · Flask · PostgreSQL 17 + pgvector · Claude (Anthropic) · Ollama (local, optional)
Context	Hackathon project, AltaSoft

Contents

1. Summary
2. The problem and the approach
3. System architecture
4. How a test run works
5. Data store and privacy
6. The AI layer
7. Feature: AI run summaries (Phase 1)
8. Feature: standards → rules → human review (Phase 2)
9. Feature: test scenarios from approved rules (Phase 3)
10. Feature: the Ask agent (Phase 4)
11. Payment initiation — PIS (Phase 5)
12. Monitoring (Phase 6)
13. Evaluation: Claude vs the local model
14. Portal and command line
15. Quality: tests, verification, bugs found
16. Current state in numbers
17. What is left
18. Key decisions
19. How to run it

1. Summary

The **API Test Tool** (`aqt`) checks whether a bank's open-banking API (XS2A, Berlin Group NextGenPSD2 as profiled for Georgia) behaves the way the standard requires. It acts like a real third-party app (TPP): mutual TLS with a QWAC certificate, every request signed with a QSealC (`Digest` + `X-Jws-Signature`), OAuth2 with PKCE for customer authentication, and every response checked against the standard. Tests are YAML scenarios; banks are YAML profiles; a built-in **MockBank** lets everything run without a real bank. It now covers both halves of open banking: **account information** (AIS) and **payment initiation** (PIS).

The **AI layer** on top of it is built so that every AI output is traceable and checked:

- **Store everything** (Phase 0): runs, scenarios, steps, HTTP exchanges and events in PostgreSQL, masked before writing; every model call audited.
- **AI run summaries** (Phase 1): a triage of each run — causes, who is at fault, evidence, clause, next step, what changed.
- **Rules from the standards** (Phase 2): the AI drafts rules from a clause with the exact sentence they rest on; code verifies every quote; a person approves.
- **Test scenarios from approved rules** (Phase 3): the AI writes a scenario; code loads it, runs it on MockBank, feeds errors back; a person approves; a coverage view links clause → rule → scenario → result per bank.
- **Ask agent** (Phase 4): staff ask questions in plain language; the agent answers with read-only tools and its citations are checked in code.
- **Payment initiation** (Phase 5): MockBank payments and a `pis` suite, gated to sandbox test accounts.
- **Monitoring** (Phase 6): scheduled runs, regression alerts (+ webhook), trends, retention.
- **Eval sets**: fixed cases with known answers for every AI feature, scored in code — and a measured comparison of Claude with the local model (chapter 13).

> **Design principle.** Facts are computed in code; judgement comes from the model; a person approves. Counts, run-to-run changes, ids, quotes, dry-run results and alerts are computed or checked in code. The model groups, explains and drafts. Nothing the AI drafts becomes a rule or a test until a human approves it, the test engine contains no AI, and the AI is graded by code, never by another model.

Status at a glance

Phase	What it delivers	Status
0 · Foundations	PostgreSQL store, masking, import, provider layer with audit log	done
1 · Run summaries	AI triage in CLI and portal; validated in code	done
2 · Standards + rules	4 standards → 379 clauses, hybrid search, rule drafting, quote checks, review workflow	done
3 · Scenario drafting	Rule → scenario → engine check → MockBank dry run → review; coverage	done
4 · Ask agent	Questions over runs, alerts, rules and standards; read-only tools; checked citations	done
5 · PIS	MockBank payments, `pis` suite, polling, test-account gate	done

6 · Monitoring	Schedules, alerts + webhook, trends, retention	done
Eval sets	18 cases across the 4 AI features; Claude vs local measured	done

147
automated tests, all passing

35
scenarios · 28 pass on MockBank

18/18
eval cases passed by Claude

14/18
eval cases passed by the local model

2. The problem and the approach

The problem

- Writing conformance tests by hand from a 341-page standard (plus Georgian profiles) is slow, and it is hard to say which requirements are covered.
- Test results are long and technical; working out *why* a run failed, and whether it is the bank or our side, takes an expert.
- History lives in report folders; nobody can ask "when did this bank start failing?", and nobody notices when it does.

The approach — and why it is shaped this way

Original idea	What we built, and why
AI writes tests directly from the documents	**Document → approved rule → test.** A one-line rule with its quote can be reviewed in seconds; every generated test traces back to a sentence of the standard and is proven on MockBank before a person sees it.
AI "tests the APIs"	**The AI writes, the engine runs.** The model never calls a bank; results stay reproducible.
AI summaries	**Triage, not prose:** grouped by cause, bank vs our side, evidence, clause, next step; facts computed in code.
Rule drafting with human approval	Kept and strengthened: quotes are **verified in code**; a failed quote goes back to the model once.
Store every data / monitoring	Kept, with **masking before writing**, an audit log, and alerts computed in code after every run.
RAG agent over logs	**An agent with query tools** (counts and time ranges need SQL, not similarity search); its SQL runs under a read-only database login.

Model choice: measured, not assumed

Every feature defaults to **Claude** (`claude-opus-5-5` , now verified live). A **local model** (`qwen3:8b` via Ollama) stays available as an opt-in for offline use or when no data may leave the machine. The eval sets measured both on the same 18 cases: Claude passed **18/18** (100% of checks), the local model **14/18** (93%), and Claude was about **4× faster** in total. Embeddings for search are always local (`bge-m3`).

3. System architecture

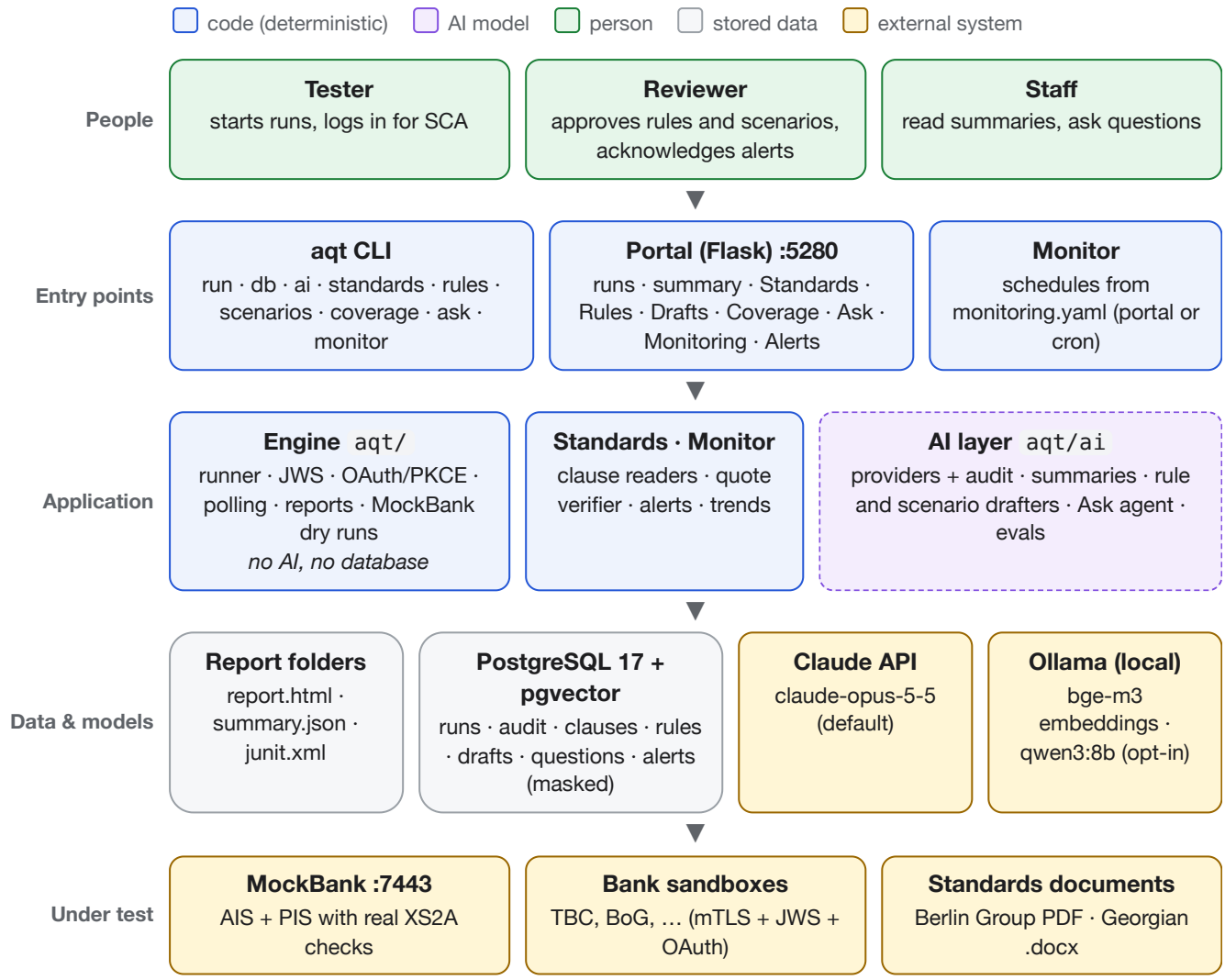


Figure 1 — Components. The engine has no AI and no database dependency; everything that reaches the database or a model is masked first.

Packages

Package	Responsibility	Lines
`aqt/` (engine)	YAML binding, runner, HTTP/TLS/SCA steps with polling, JWS, OAuth/PKCE, reports, MockBank dry runs	3 641
`aqt/cli.py`, `cli_ai.py`	The `aqt` command, all sub-commands, `.env` loading	1 170
`aqt/mockbank/`	Fake bank: consents, accounts, payments (lifecycle, table-4 checks), OAuth/PKCE	1 427
`aqt/portal/`	Flask UI incl. Standards, Rules, Drafts, Coverage, Ask, Monitoring, Alerts	2 569
`aqt/store/`	Schema (4 SQL migrations), masking, import, queries, coverage, agent views	1 948
`aqt/standards/`	PDF and Word clause readers, quote verifier, sources manifest	452
`aqt/ai/`	Providers + audit + tool calling, summaries, rules, scenarios, Ask agent, evals; prompts as files	2 650

`aqt/monitor/`	monitoring.yaml, due times, alerts and webhook, trends	247
`tests/` · `evals/`	pytest (147 tests) · eval cases (18)	2 060 · 464

Technology

Runtime	Python 3.11+ (developed on 3.14); Flask portal; PyYAML, cryptography, pyOpenSSL
Database	PostgreSQL 17 with pgvector (Docker, port 5435); psycopg 3, plain SQL, numbered migrations
AI	Official `anthropic` SDK (Claude, beta endpoint with refusal fallback); Ollama over HTTP for embeddings and the optional local model
Install	Core: `pip install -e .[test]` ; store + AI: `pip install -e .[ai]` ; API key in a git-ignored `.env`
Constraint	Standalone: only public PyPI packages; no dependency on AltaSoft's internal repositories

4. How a test run works

A run takes a bank profile and a set of scenarios. **Preflight** scenarios (certificates, TLS, OAuth discovery, a signed request) always run first; if a critical one fails, the rest are marked *Blocked* instead of failing one by one for the same reason. Each scenario is a list of steps; each request is signed, sent over mutual TLS, and its response is checked (status, error codes, headers, JSON fields, signature).

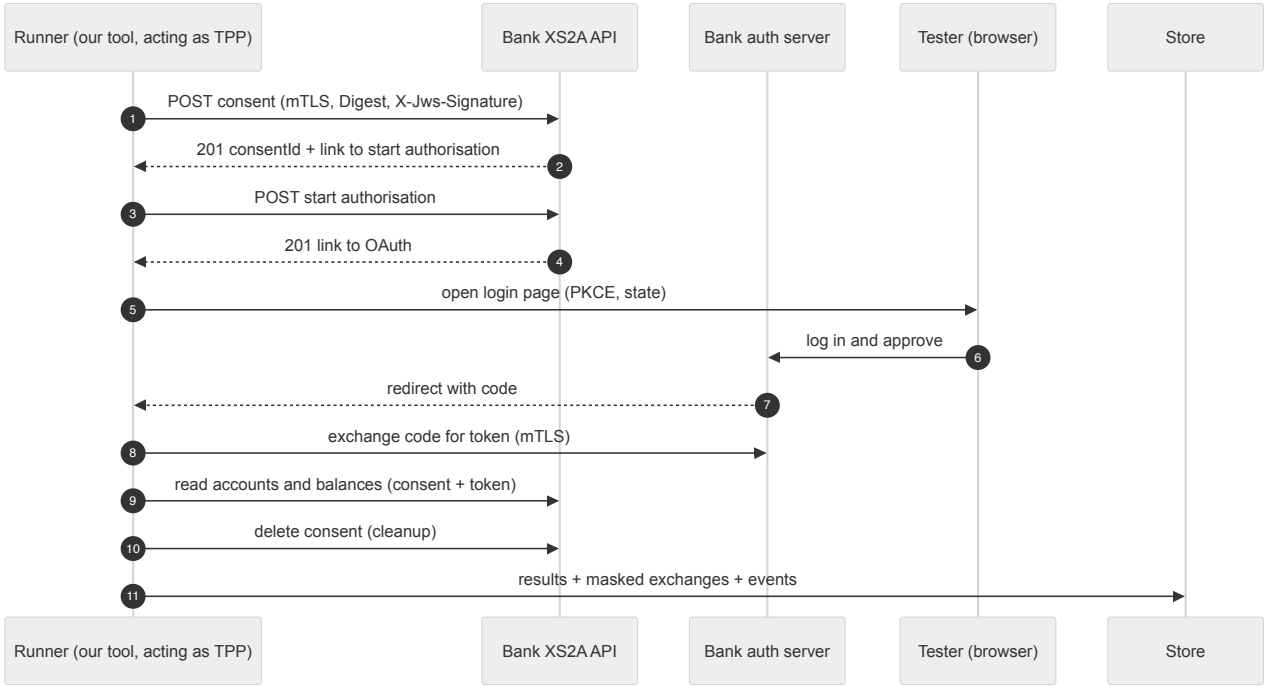


Figure 2 — An account-information scenario (AIS-001) from start to store.

Outcome	Meaning
Passed	All expectations met.
Failed	The bank's API did not behave as expected (a bank-side deviation).
Error	A problem on our side: configuration, certificates, timeout waiting for a login.
Blocked	Not run because a critical preflight check failed.
Skipped	The bank does not declare the feature (expected; not a problem).

Requests can **poll** (repeat until a status is final, e.g. a payment settling) and expectations can accept **one of several** values. **Unattended** runs (monitoring) skip scenarios that need a person to log in, unless the login is automatic (MockBank).

Reports are written as before (`report.html` , `summary.json` , `junit.xml`), and the run is also saved to the database. A database problem never fails a run: it becomes a notice, and the folder can be imported later.

5. Data store and privacy

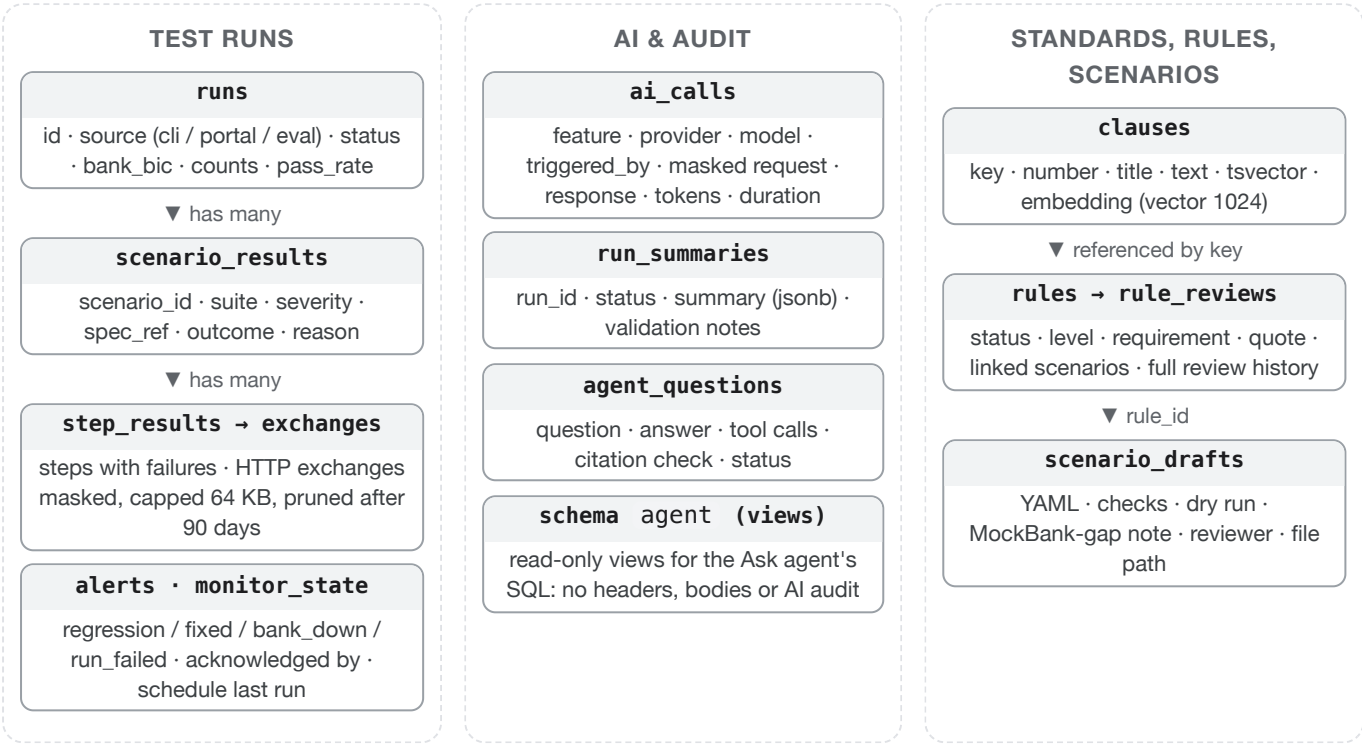


Figure 3 — Database tables (PostgreSQL, 4 migrations), grouped by purpose; key columns only.

Masking: what never reaches the database or a model

Data	Treatment
Authorization, Cookie, PSU-ID headers	Value replaced (`Bearer ***(masked)`)
Tokens, codes, secrets in JSON, forms, URLs	`access_token` , `refresh_token` , `code_verifier` , … masked at any depth; the OAuth `code` in forms and query strings
Credentials written in free text	`Bearer …` / `Basic …` in error messages, alerts and JSON strings masked (added in Phase 6)
XS2A error codes	**Kept readable** (`"code":"FORMAT_ERROR"` is evidence, not a secret)
IBANs	Country, check digits and last 4 kept: `GE29**************4917`

The Ask agent's SQL cannot escape. It logs in as its own database role `aqt_agent` , which can only read the `agent` views: one prepared statement, read-only, 5-second timeout. A first design switched role inside the main connection; probing showed the SQL could switch back (`set_config('role', …)` , a second statement), so it was replaced. The database now refuses every escape attempt, and a test proves it.

Audit

Every model call — summaries, drafting, the Ask agent's tool rounds, eval runs — is a row in `ai_calls` (feature, provider, model, who, masked request, answer, tokens, duration, prompt hash). Rule decisions are rows in `rule_reviews` , scenario approvals link the rule, alerts record who acknowledged them.

6. The AI layer

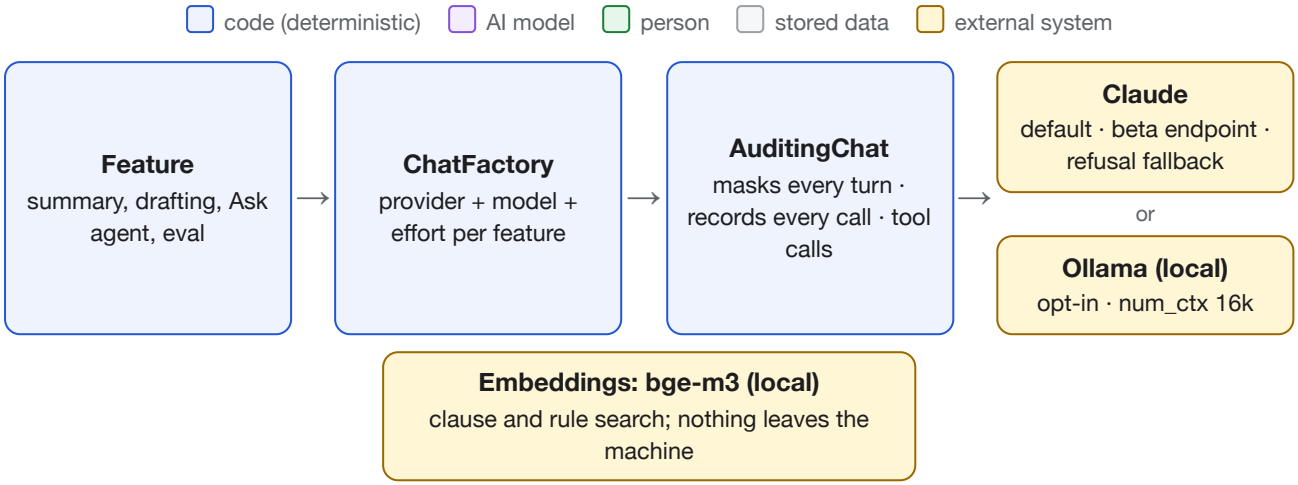


Figure 4 — Every model call goes through the same wrapper. Tool calling (Ask agent) uses one provider-neutral message format.

Feature	Default	Notes
Run summary	Claude, effort medium	Local opt-in (button, `--provider ollama`)
Rule drafting	Claude, effort high	One repair round for quotes the verifier rejects
Scenario drafting	Claude, effort high	Up to 3 attempts with engine errors and MockBank results fed back
Ask agent	Claude, effort medium	Up to 8 tool rounds; read-only tools
Embeddings	bge-m3 (local)	Multilingual: English questions find Georgian clauses

Safeguards in the provider layer

- **Refusal fallback:** our prompts contain security-testing material; a declined request is re-run on Anthropic's recommended fallback model.
- **Tool calling:** Claude's own blocks (including thinking) are sent back unchanged in a tool loop, as the API requires; tested against a fake server.
- **Context window and timeouts** for the local model; timeouts are audited failures.
- Prompts and their JSON schemas are files in `aqt/ai/prompts/`, reviewed like code and measured with `aqt ai eval` before and after a change.

> **Claude verified live** (25 October): `aqt ai check` answers in about 2 s through the beta endpoint with the refusal fallback, and the full eval set ran against it (35 calls, 130,607 input / 23,715 output tokens). The key comes from a git-ignored `.env`; values are never printed.

7. Feature: AI run summaries (Phase 1)

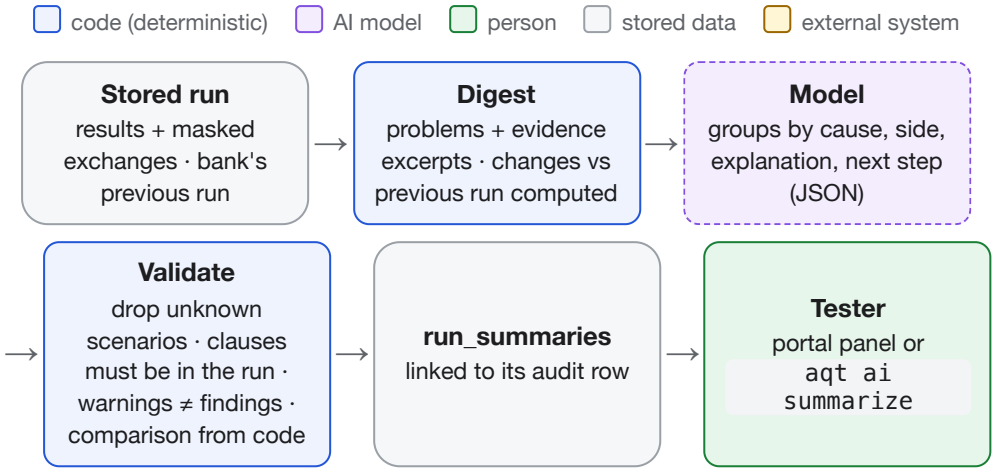


Figure 5 — From a stored run to a checked triage.

A summary contains: a **health** verdict (healthy / degraded / broken / inconclusive), **findings** grouped by root cause, each with the side at fault (bank, our side, environment, unclear), evidence quoted from the responses, the clause, and a next step; the change since the previous run; and warnings worth a look.

Evaluation on real data

All 9 stored runs with results were summarized with the local model and compared with the database. Five kinds of error were found and each was fixed — preferably in code:

Error found	Fix
"Since the previous run" described changes wrongly	Now written **in code** from the computed changes
Warnings of passed scenarios reported as problems	**Code** moves them to "worth a look"; findings about skipped scenarios are dropped (Python port)
Random 503s of the TBC mock sandbox blamed on the bank	Prompt: temporary 5xx = environment unless persistent
A tester clicking "Deny" treated as a critical bank fault	Prompt: `access_denied` at login = unclear, suggest a re-run
A correct but shortened clause reference was removed	Validator matches clause numbers and restores the run's exact wording
Our expired certificate blamed on the bank (found by the eval set)	**Code** marks local certificate checks as ours in the digest; prompt: certificate/signature problems are our side

> After the fixes, re-run on the 5 affected runs: all correct against the data. The TBC mock's deviation (global consent rejected with `FORMAT_ERROR` instead of `PARAMETER_NOT_SUPPORTED`) is found every time with the right clause. Local speed: 15–35 s per run.

8. Feature: standards → rules → human review (Phase 2)

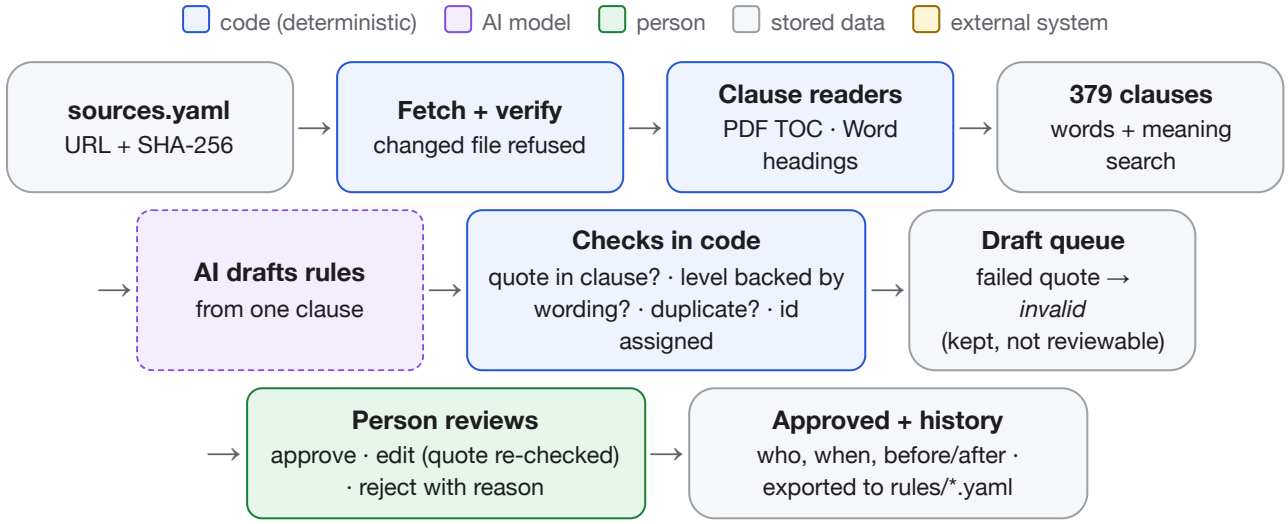


Figure 6 — The rule pipeline. Only a person moves a rule to approved.

The standards

ID	Document	Version	Language	Clauses
`BG-IG`	Berlin Group NextGenPSD2 XS2A Implementation Guidelines	1.3.16 (Dec 2025, latest)	English	182
`GE-XS2A`	Georgian XS2A Implementation Guide (openfinance.ge)	0.8.RC3	Georgian	133
`GE-JWS`	Georgian JSON Web Signature Guide	0.5.RC1	Georgian	32
`GE-IAA`	Georgian Identification, Authentication and Authorization Manual	0.8.RC1	Georgian	32

- **PDF:** the table of contents gives the clause list; each entry is found in order in the body; deeper sections (e.g. 6.3.1.2) are found inside their parent. Running headers and footers are dropped by pattern (pdfplumber puts footers last). A defect in the PDF (a stray sentence glued into the chapter 6 heading) is corrected in the manifest. The Python reader gives the same 182 clauses as the C# one; every existing rule quote still verifies.
- **Word:** heading styles give the structure; numbers are computed like Word. Verified against the published PDF of the Georgian guide, because the document's own cross-references contradict each other.
- The Georgian guides exist only in Georgian. The Georgian text is the source of truth; each rule carries an English translation of its quote, labelled as an AI translation.
- Documents are downloaded, not committed; a changed file (different SHA-256) is refused until reviewed.

What a rule looks like

```
GE-CON-002   MUST · AIS · GE-XS2A 9.1.1.10
Requirement  When validUntil is 9999-12-31, the bank sets the consent's validity to the maximum
             allowed (at most 90 days) and returns that date in the consent details.
Quote        "გამონაკლისს შეადგენს მნიშვნელობა „9999-12-31“, რომლის მიღების შემთხვევაში აუცილებელია
      ამსმპ-მა თანხმობის ვადა შეცვალოს მაქსიმალურ დასაშვებ მნიშვნელობაზე (არაუმეტეს 90 დღისა) ..."
EN (AI)      "The exception is the value '9999-12-31' ... must be replaced with the maximum
allowed
             duration (≤90 days) ..."
Check        positive — create consent with 9999-12-31, read it back, expect ≤ 90 days
Tool checks  ✓ quote found in the clause
```

Checks done in code before a person sees a draft

Check	Effect
Quote occurs in the clause	Ignores only spacing, line breaks, hyphenation, quote/dash style and case; "…" marks skipped words. Fails → rule stored as **invalid** (kept, never reviewable).
Repair round	Quotes that fail the check go back to the model **once** with the verifier's reason ("copy exactly or remove"); the new answer is checked again. Added after the eval set showed the local model rebuilding table rows ("Code 202: …") that the standard does not contain.
Level vs wording	MUST/SHOULD/MAY must be backed by the quote's keywords (English and Georgian); otherwise a warning for the reviewer.
Duplicates	Close in meaning (embeddings) **and** naming the same fields, codes and statuses; otherwise "accept availableAccounts" and "accept allPsd2" looked identical.
Review	Approve only drafts; reject needs a reason; an edited quote is checked again; a stale update is refused; every action recorded with the reviewer's name.

Findings from the first drafts (local model). It finds requirements and quotes them exactly, including Georgian — it found the Georgia-specific **90-day consent limit** (`PERIOD_INVALID`), which the Berlin Group guide does not have. Its weak point is the level: it marks optional functions as MUST; the level check flags this to the reviewer. In clause 4.1 it joined non-adjacent bullet items without "…": the quote check correctly rejected 8 such drafts (the prompt now forbids it). On GE-XS2A 9.2.2 it changed one Georgian word inside a quote (სათაურის → სათავის), and the check caught that too.

9. Feature: test scenarios from approved rules (Phase 3)

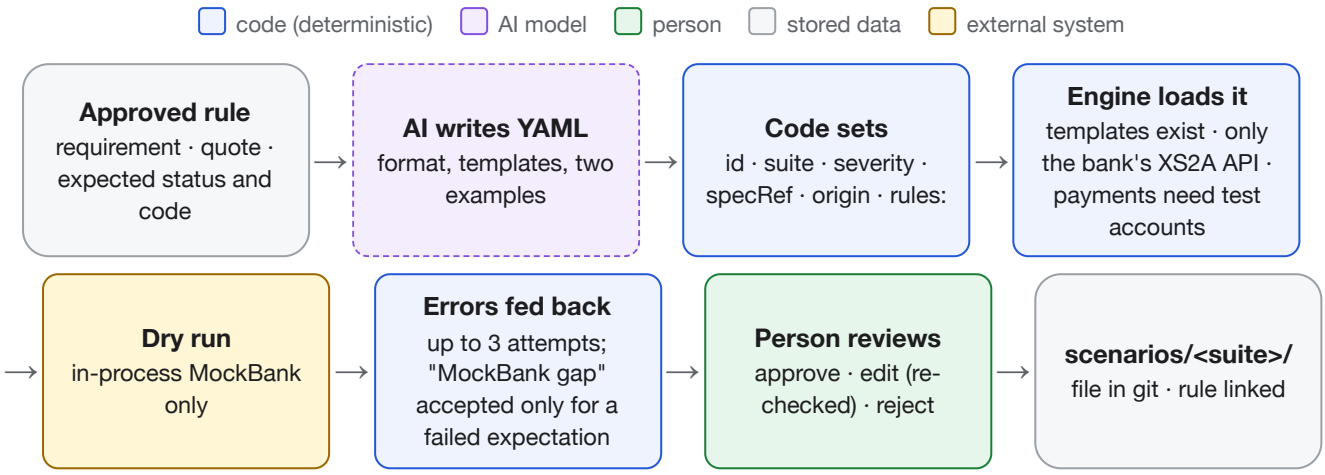


Figure 7 — From an approved rule to a test in the repository. Drafts never run against a real bank.

When a draft fails on MockBank, the model may answer that the scenario is right and **MockBank does not implement the rule**. That claim is accepted only when an expectation failed (never for a tool error) and is shown to the reviewer. The first live draft did exactly this: GE-CON-001 (90-day consent limit) failed because MockBank had no such limit — so the limit was added to MockBank, and the same draft then passed on the first attempt.

The **Coverage** page and `aqt coverage` show, per approved rule: its clause, the scenarios that verify it, and their last result on each bank — computed in code from the rules, the `rules:` field of the scenario files and the stored runs. `aqt validate` warns when a scenario cites a rule that is missing or not approved.

10. Feature: the Ask agent (Phase 4)

Staff ask in plain language — "Why did TBC fail on 21 October, and has it recovered?", "Which scenarios fail most often?", "Which error code must a Georgian bank return for a consent over 90 days?" — on the **Ask** page or with `aqt ask`.

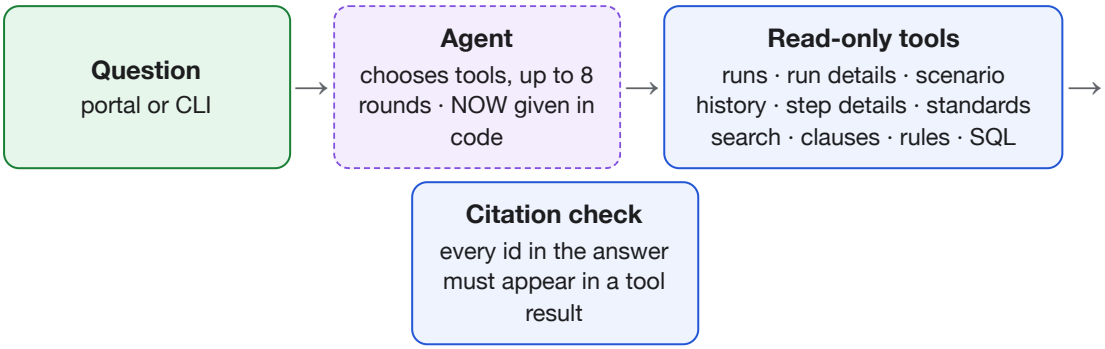


Figure 8 — The Ask agent. Tool results are masked; the answer and its tool calls are saved.

Safeguard	How
Cannot change anything	Every tool reads. The SQL tool logs in as `aqt_agent` (views only, read-only, one statement, 5 s) — see chapter 5.
No invented ids	Run ids, scenario and rule ids and clause keys in the answer are checked against what the tools returned; unknown ones are listed and the answer is marked *unverified*.
No "nothing happened" guesses	Found live: the local model said TBC had not recovered after limiting its own search to before the 22nd. Fixed in code (a run's details now list the bank's later runs) and in the prompt.
Traceable	Each question is stored with its answer, the tools it called and their results; each model round is in `ai_calls`.

11. Payment initiation — PIS (Phase 5)

Built from chapter 8 of the Georgian XS2A guide, read first: products `domestic` and `foreign` (mandatory) and `aspsp` (within the bank); initiation, status, details, cancellation, OAuth SCA with scope `PIS:<paymentId>`.

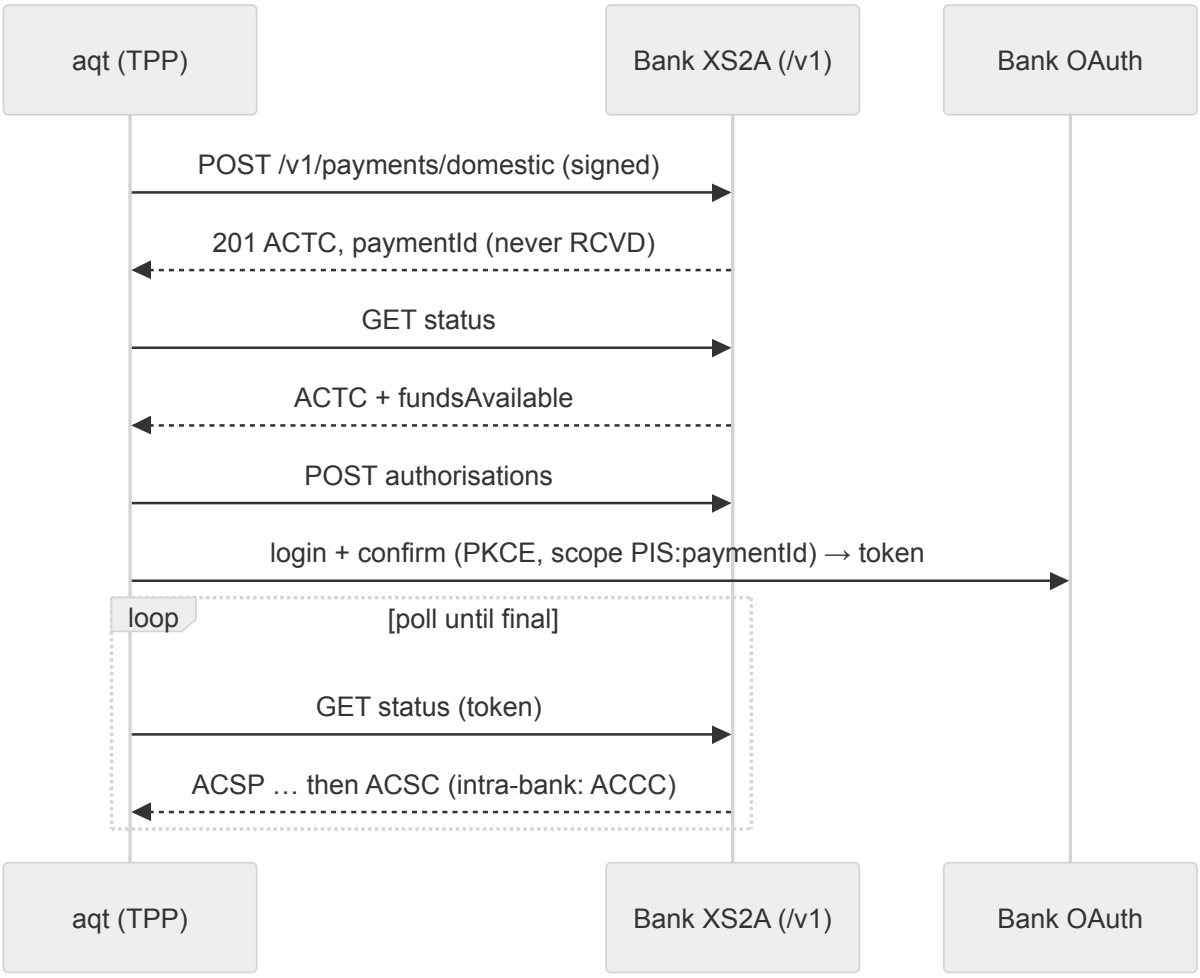


Figure 9 — A domestic payment (PIS-001), including polling until it settles.

Scenarios	What they check (clause)
PIS-001 … PIS-005	Domestic payment end to end (8.3–8.10) · intra-bank payment ends in ACCC (8.10) · cancel before authorisation (8.7) · cancelling a booked payment → 405 CANCELLATION_INVALID · foreign payment accepted with `fundsAvailable` (8.5)
NEG-PIS-001 … 007	Field the product never allows · invalid IBAN · missing amount (table 4 → FORMAT_ERROR) · unknown product (PRODUCT_UNKNOWN) · status without the token after SCA · insufficient funds at initiation · foreign payment without creditor name

A payment moves money, so it is gated twice. PIS scenarios run only when a bank profile names its **sandbox test accounts** (`variables.pis`) — the TBC mock has none and skips the whole suite. AI-drafted scenarios that send payment data without that requirement are refused in code; a live draft that slipped past the first version of this check (payment data sent to a signing-basket path) led to the check being widened and tested.

Also found: MockBank's sandbox IBANs had invalid check digits — a real bank would have rejected them. Not built: bulk and periodic payments, signing baskets.

12. Monitoring (Phase 6)

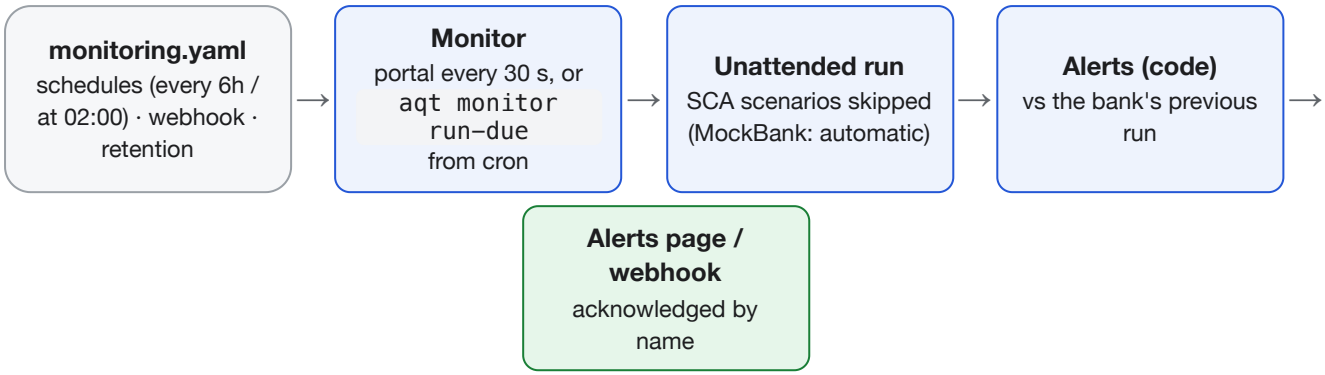


Figure 10 — Monitoring. Alerts are computed after every saved run — from the CLI, the portal or a schedule.

Alert	When
regression	A scenario passed in the bank's previous run and fails now
fixed	It failed before and passes again
bank down	A critical preflight check started failing — reported once, not on every run while the bank stays down, and not doubled as a regression
run failed	The run itself could not complete

The **Monitoring** page shows schedules (last run, next run, Run now), the pass rate per bank over 30 days, flaky scenarios and the slowest endpoints. On the real history it showed TBC's PRE-003/004 as flaky (the 21 October outage) and TBC's consent-status call as the slowest endpoint (1.9 s on average, 8.7 s at worst). Retention deletes stored HTTP exchanges after 90 days; results, summaries and alerts are kept.

13. Evaluation: Claude vs the local model

Each AI feature has a fixed eval set (`evals/*.yaml`) with known answers. `aqt ai eval` runs every case through the **real** feature in a temporary database (the real store is never touched) and grades it with **checks in code** — no model judges another model. Both providers ran the same 18 cases with the same prompts on 25 October.

Eval set	Cases, and what is checked
Run summaries (4)	A sandbox outage, our expired certificate, a bank regression, an all-green run · health, findings for the right scenarios on the right side, nothing reported for passed or skipped scenarios
Rule drafting (4)	Three Georgian clauses (90-day consent, payment status, cancellation) and one informative clause where the right answer is "no rules" · count, required topics and levels, every quote found in the clause
Scenario drafting (4)	Three rules MockBank enforces (incl. a payment) and one it does not · dry-run outcome, YAML content, attempts, MockBank-gap judgement
Ask agent (6)	Counting, most frequent failure, a cause, recovery after an outage, a standards question, alerts · facts in the answer, every cited id looked up, a tool used

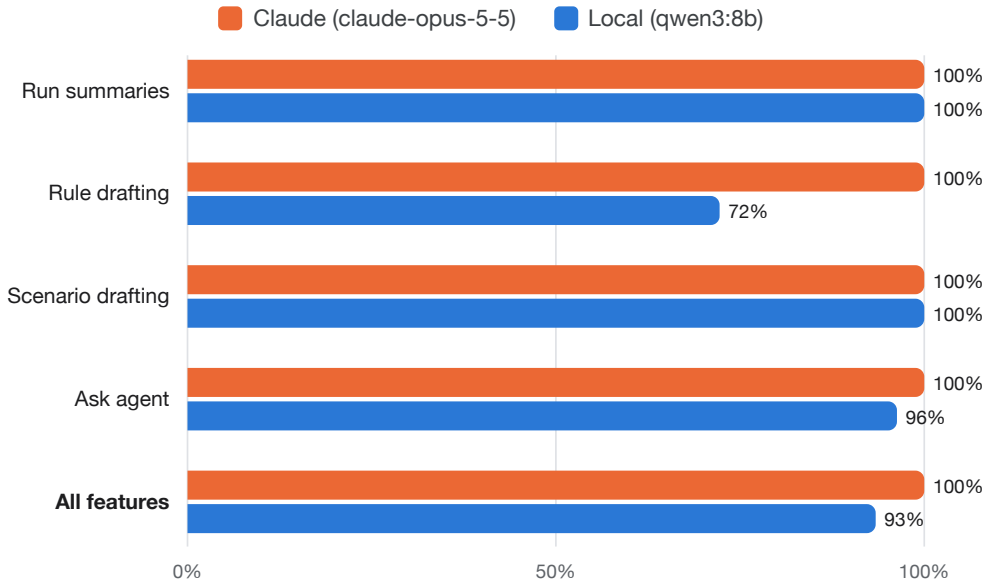


Figure 11 — Share of eval checks passed per feature (same cases, same prompts).

Feature	Claude (claude-opus-5-5)			Local (qwen3:8b)		
	Cases	Checks	Time	Cases	Checks	Time
Run summaries	4/4	25/25	25 s	4/4	25/25	78 s
Rule drafting	4/4	20/20	85 s	1/4	13/18	339 s
Scenario drafting	4/4	21/21	38 s	4/4	21/21	138 s
Ask agent	6/6	27/27	69 s	5/6	26/27	238 s
All	**18/18**	**93/93**	**217 s**	**14/18**	**85/91**	**793 s**

Run-to-run variation. The same local model passed 3 of 4 rule-drafting cases in an earlier run and 1 of 4 in this one: its rule drafting varies a lot between runs (missed duties, quotes that still fail after the repair round). Claude passed every case in both of its runs. Results above are one full run per model; the local run's rule, scenario and Ask cases were repeated once because the Ollama server crashed under load during the first attempt (an infrastructure failure, not a model result).

Model calls: Claude 35 (130,607 input / 23,715 output tokens); local 45 (87,586 / 40,414). Times are wall-clock per eval set on a MacBook; the local model ran on the same machine.

Case by case

Feature	Case	Claude	Local	Time C · L
Run summaries	`tbc-sandbox-outage`	pass	pass	7 s · 21 s
Run summaries	`our-qsealc-expired`	pass	pass	7 s · 23 s
Run summaries	`bank-regression`	pass	pass	9 s · 18 s
Run summaries	`all-green`	pass	pass	2 s · 16 s
Rule drafting	`ge-consent-period`	pass	fail a rule about 9999-12-31	18 s · 87 s
Rule drafting	`ge-payment-status`	pass	fail every quote is in the clause; a rule about fundsAvailable; a rule about JSON/application/json	23 s · 164 s
Rule drafting	`ge-payment-cancellation`	pass	fail a rule about 405	42 s · 79 s
Rule drafting	`ge-informative-examples`	pass	pass	3 s · 10 s
Scenario drafting	`consent-period-90-days`	pass	pass	10 s · 35 s
Scenario drafting	`frequency-per-day-limit`	pass	pass	10 s · 30 s
Scenario drafting	`pis-forbidden-field`	pass	pass	4 s · 20 s
Scenario drafting	`mockbank-gap-one-off-consent`	pass	pass	14 s · 54 s
Ask agent	`count-runs`	pass	pass	7 s · 48 s
Ask agent	`most-failing-scenario`	pass	pass	14 s · 68 s
Ask agent	`why-bog-failed`	pass	pass	13 s · 34 s
Ask agent	`recovery-after-outage`	pass	pass	14 s · 41 s
Ask agent	`standard-period-invalid`	pass	fail mentions GE-CON-001	10 s · 26 s
Ask agent	`alerts-on-tbc`	pass	pass	11 s · 22 s

What the evals found, and what changed

Finding	Change
Local model blamed the bank for our expired certificate	Code marks local-certificate steps as ours in the digest; prompt rule on certificate/signature problems. Passed 3/3 re-runs.
Local model rebuilt table rows as "Code 202: …" in quotes	Rejected quotes go back to the model once with the reason (repair round), then checked again.

Three eval cases contradicted the prompt (blocked scenarios, "inconclusive")	Cases corrected — Claude's "inconclusive" for our own expired certificate is right by the prompt's definition.
Two evals in parallel collided on the shared database role	Migrations retry that specific conflict; 12 databases created in parallel now succeed.

Decision. Claude is the default for every feature: it passed 18 of 18 cases (100% of checks) and was about 4× faster. The local model passed 14 of 18 (93%); its weak spots are attributing blame, quoting tables exactly and citing the right ids in multi-step questions. It stays as an opt-in for offline use or when no data may leave the machine. Ollama itself remains required for the local `bge-m3` embeddings behind search.

Run it again after any prompt change: `aqt ai eval --provider anthropic` and `--provider ollama`; reports in `reports/evals/`.

14. Portal and command line

Portal (Flask, http://localhost:5280)

Page	What it does
Dashboard · Runs · Run	Banks with last result; start runs, watch live, SCA login button; steps and HTTP exchanges; AI summary panel
Standards · Rules	Clause outline and search (English or Georgian); rule drafting; review queue: approve / edit / reject / reopen; export
Drafts	Scenario drafts from approved rules: YAML, dry-run table, MockBank-gap note; approve writes the file; edit is re-checked
Coverage	Approved rules → scenarios → last result per bank; per-service counts (AIS / PIS / COMMON)
Ask	Questions answered in the background; history with what the agent looked up and the citation check
Monitoring · Alerts	Schedules, pass-rate lines, flaky scenarios, slowest endpoints · alerts acknowledged by name

Command line (`aqt`)

Command	Purpose
`run --bank X [--suite ..] [--with-mock] [--sca-auto] [--unattended]`	Run scenarios; saved to the store and checked for alerts when a database is set
`db migrate | import | status | prune`	Schema, import, counts, retention
`ai check | summarize | eval`	Provider check; AI triage; eval sets
`standards …` · `rules …`	Fetch, ingest, search clauses · draft and review rules
`scenarios draft | drafts | show | approve | reject` · `coverage`	Scenario drafts from approved rules · coverage
`ask "…" | ask history`	The Ask agent
`monitor status | run-due | alerts | ack | trends`	Monitoring
`validate` · `list` · `mock` · `portal`	Check all YAML; list; run MockBank; run the portal

15. Quality: tests, verification, bugs found

Test file	Tests	Covers
Engine (5 files)	35	YAML, variables, JSON paths, signing, reports, every shipped YAML, MockBank end to end (AIS + PIS), polling, 90-day limit, payment rules, `.env`
`test_store.py` · `test_standards.py`	27+	Masking, import, PostgreSQL round trips, search; readers and quote verifier
`test_ai.py`	22+	Exact Claude request incl. tools (fake server), audit, summaries, drafting and repair round, review workflow
`test_scenario_drafting.py`	11	Code-owned fields, safety checks, MockBank dry runs, MockBank gap, review, coverage
`test_agent.py`	7	Message formats, citation check, read-only SQL escape attempts, agent loop and round limit
`test_monitor.py`	16+	monitoring.yaml, due times, alerts, webhook masking, trends, retention, monitor, pages
`test_evals.py` · `test_portal_ai.py`	8 · 6	Case files well-formed, scoring separates right from wrong · portal pages

147 tests pass with `AQT_TEST_DB` (a throw-away database per test); without a database 111 pass and the rest are skipped. MockBank run: 28 passed, 7 skipped (TBC-only), 0 failed.

Bugs found by testing against real data or by the evals, and fixed (since the previous overview)

Bug	Effect if left
Ask agent's SQL could switch back to the main database user	The agent could read or change anything
MockBank had no 90-day consent limit	A correct generated test failed on MockBank
MockBank sandbox IBANs with wrong check digits	Payment tests rejected by a real bank's checks
Payment-data check missed signing-basket paths and templates	An AI draft could move money without test accounts
Bank down reported twice (also as a regression)	Duplicate alerts
"Bearer …" in free text not masked	Tokens in alerts, messages and prompts
Agent claimed "not recovered" without looking	Wrong answers to "has it recovered?"
Our expired certificate blamed on the bank (local model)	Wrong side in the triage
Table rows paraphrased in rule quotes	Valid rules discarded as invalid
Parallel migrations collided on the shared role	Evals or test runs failing when run together

Every step, decision and finding is recorded in `NOTE.md` (49 entries); `ARCHITECTURE.md` holds the current diagrams.

16. Current state in numbers

Item	Value
Scenarios	37 files: preflight 4 · negative 17 (consents, signatures, payments) · AIS 2 · PIS 5 · TBC 7 · templates
MockBank run	28 passed · 7 skipped (TBC-only) · 0 failed
Runs in the store	16 (MockBank 11, TBC 4, Bank of Georgia 1), 206 scenario results, 333 HTTP exchanges, 100 events
Standards / clauses	4 documents, 379 clauses, all with embeddings
Rules	**4 approved** · 11 drafts to review · 9 invalid (quote not verbatim)
AI calls audited	89 in the main store (12 Claude, 77 local) plus the eval runs
Automated tests · eval cases	147, all passing · 18
Git	Everything merged into `main` (25 October); the work branches were deleted

17. What is left

Every phase of the AI plan and its eval sets are complete. What remains is mostly for people, plus work outside the original plan.

For the team

Item	Why
Approve account-information rules	GE-CON-001 (after the edit in the review guide), GE-CON-002, GE-AUTH-001 — then the Drafts and Coverage pages have real content.
Real bank sandboxes	Certificates, credentials and PIS test accounts for each bank profile; first runs with manual SCA.
Alert webhook	Set `AQT_ALERT_WEBHOOK` to a Slack/Teams channel.

Open items and risks

Item	Detail
before staff use Login and roles	The portal has no login; the reviewer name is typed in. Real staff review needs authentication and an approver role.
Data policy	Masked run data goes to Claude for summaries and the Ask agent. Confirm for real bank data, or switch those features to local.
Eval sets are small	18 cases; extend with real staff questions and real failures as they occur.
English Georgian guides	Only Georgian versions exist; translations are AI-made per rule.
Continuous integration	No CI yet (pytest with a PostgreSQL service on each push; evals on prompt changes).

Outside the AI plan

Bulk and periodic payments, signing baskets, decoupled/embedded SCA modes, client registration, OpenAPI schema validation, Excel report, `aqt compare`.

18. Key decisions

Decision	Choice and reason
Language	Python (team decision; the teammates ported the engine, verified both ways against the C# client and MockBank). The AI layer was ported to match.
Database	PostgreSQL (Docker) with pgvector: one database for runs, audit, clauses and vectors. Python uses plain SQL on the same schema; the database the C# tool created is kept.
Models	Claude by default for every feature (measured: chapter 13); local Ollama model opt-in only; local embeddings.
Local model	`qwen3:8b` when chosen (5× faster than `gemma4:26b`); kept for offline / no-data-leaving use.
Scenario drafts	Code owns ids and links to the standard; MockBank-only dry runs; approval writes the file.
Ask agent	Read-only tools; SQL under its own database login; citations checked in code.
Payments	Gated to sandbox test accounts named in the profile; enforced in code for AI drafts.
Monitoring	Schedules in YAML (versioned); alerts in code; unattended runs never wait for a login.
AI quality	Eval sets graded in code, never by a model; run before and after every prompt change.
Standards	Latest versions: Berlin Group IG 1.3.16; Georgian guides from openfinance.ge (Georgian only).
Translation	No whole-document translation; Georgian text is the source of truth; per-rule English translation labelled as AI.
Rules storage	Database is the source of truth (queue, history, search); approved rules exported to YAML for git.
Approval	The team approves now; real staff later (needs login). Every action recorded with name, time and reason.
Scope	AIS first; PIS designed in from the start, built in Phase 5.
Repository	Public GitHub repository; source documents not committed (licences); no secrets committed.

19. How to run it

```
python -m venv .venv && source .venv/bin/activate
pip install -e ".[test,ai]"
docker compose up -d                                    # PostgreSQL + pgvector on
localhost:5435
aqt db migrate
aqt db import reports

# a full test run against the built-in fake bank, saved to the database
aqt run --bank MOCK --with-mock --sca-auto --db
"Host=localhost;Port=5435;Database=aqt;Username=aqt;Password=aqt_dev"

# AI: put ANTHROPIC_API_KEY=... in .env (git-ignored); Ollama with bge-m3 for search (qwen3:8b
optional)
aqt ai check
aqt ai summarize latest
aqt ai eval --provider anthropic                        # eval sets; reports in reports/evals/

# standards and rules
aqt standards fetch && aqt standards ingest
aqt rules draft GE-XS2A:0.8.RC3:9.1.1.10
aqt scenarios draft GE-CON-001                          # needs an approved rule; dry run on
MockBank
aqt coverage
aqt ask "Why did TBC fail on 21 October?"
aqt monitor status                                      # schedules in monitoring.yaml

# web portal (store/AI pages on when the database is reachable)
aqt portal                                              # http://localhost:5280

# tests (including database tests)
AQT_TEST_DB="host=localhost port=5435 user=aqt password=aqt_dev" pytest
```

Further reading in the repository

`README.md`	User manual: commands, profiles, scenarios
`ARCHITECTURE.md`	Current diagrams (updated after every phase)
`docs/ai/PLAN.md`	AI plan, phases and decisions
`docs/ai/REVIEW-GUIDE.md`	How to approve, edit and reject rules
`NOTE.md`	Journal: every step with what and why
`standards/sources.yaml`	Standards: versions, URLs, hashes